AI INFRA RESEARCH SERIES

Mixture-of-Kittens

面向 NVL72 的 MoE 训练 Megakernel

分析对象

Mixture-of-Kittens: MoE Megakernel for NVL72s

作者与机构

Stuart H. Sul、Nash Brown、Henry Wildermuth、William Lin、Federico Cassano、Christopher Ré；Cursor 与 Stanford。

版本范围

arXiv:2609.36070v1；结合 Cursor 官方技术文章与公开 README。代码接口以本次核对的 README 为准，不将未来兼容性当作已验证能力。

核心判断

MoK 把专家前馈计算与 dispatch/combine 放进统一 GPU 执行框架，以通信方向、SM 分工、minibatch 流水和 ring buffer 共同减少等待。它不是 MCore，也不融合 Attention；前向路由与调度准备仍在主体 Megakernel 之外。

阅读定位：机制与公式、系统实现、实验口径、局限与复现。

说明：性能数据均标明来源；本报告没有独立训练模型或复跑 GPU 性能。推导与教学算例不作为作者实测结果。

整理日期：2026 年 10 月 31 日

阅读导航

1 先把边界讲清：MoK 不是整个 Transformer 2

2 为什么单独优化 GEMM 和通信不够 2

3 通信方向：谁主动搬运，谁知道完成 3

4 SM 分工与 minibatch 流水 3

5 backward：把 dgrad 放在关键路径，wgrad 延后 4

6 Ring buffer：动态路由不再要求 host 读取尺寸 4

7 确定性、MXFP8 与 CLC 的真实含义 5

8 性能结果及其适用条件 6

9 源码接口与调试顺序 6

10 与 MoonEP 的关系及最值得研究的实验 7

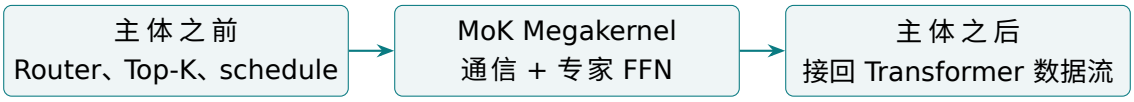
11 原始资料与结论 7

文中的 [S1]、[S2] 等编号指向文末原始资料。引用附带章节、表号或源码位置，便于核对；不同报告的编号独立。

1 先把边界讲清：MoK 不是整个 Transformer

一个典型 MoE Transformer block 可以写成 Attention → MoE FFN，另有 norm、残差等操作。MoK 集中优化后半部分中的**专家计算与跨 GPU token 通信**，不负责 QK、softmax 和 AV 的 attention 计算。[S1，Fig.1；S2]

路由投影、top-K 选择、top-K 信息汇总和 GPU schedule 构造位于主体 Megakernel 之前。它们提供所选专家、混合权重和工作计划。主体则执行 token 的搬运、shared/routed experts 的前馈，以及结果返回与合并。[S1，Fig.1；S3]



“一个 kernel” 应理解为相应 forward 或 backward 主体融合的执行边界，不是一次 launch 包办全模型训练。权重量化、schedule 建立、优化器和外层分布式训练仍有自己的操作。[S3]

MCore 则是完整训练基础设施，可以管理 Attention、MoE、参数切分和优化器。二者是不同层次；概念上 MoK 可作为某个 MoE 执行后端，但实际集成需要接口、梯度、并行组和生命周期适配。

2 为什么单独优化 GEMM 和通信不够

传统路径常表现为 dispatch 完成后才启动 grouped GEMM，专家结果完成后才启动 combine。中间穿插 permute、量化、激活和同步。即使每个局部 kernel 都很快，阶段之间仍可能有空洞。[S1，§2–3]

可以用下面的分析式理解成本：

$$T \approx \max(T_{\text{compute}}, T_{\text{HBM}}, T_{\text{network}}) + T_{\text{exposed}} + T_{\text{sync}} + T_{\text{host}}.$$

这不是适用于任意调度的精确预测公式，而是说明：理想重叠只能覆盖独立工作；依赖边界、资源争用与 CPU 空洞仍会暴露。

NVL72 提供较大的高速 scale-up 域，使小粒度 GPU 间直接访问更有意义。但网络变快以后，完成信号、kernel 边界与 CPU 参与反而可能成为显著成本。MoK 的设计是围绕这一硬件环境重新组织执行，而不是把通用 All-to-All 换一个名字。[S2]

2.1 更多 stream 不自动等于更好的 overlap

同一 token 的 dispatch → expert → combine 有真实依赖。若只是分别放进多个 stream，仍要等数据；而且 stream 不保证以期望比例占用 SM。通信和计算可能争抢执行资源，导致看似同时运行，整步反而更慢。

MoK 在主体内部控制任务与 SM 分工，使一部分 SM 负责搬运，另一部分负责计算，并在不同 minibatch 之间建立流水线。这里的 minibatch 是算子内部的工作粒度，不要与训练框架的 global batch、microbatch 混为一谈。[S1，§3；S2]

3 通信方向：谁主动搬运，谁知道完成

3.1 前向 dispatch 用 Pull

专家所在 GPU 主动从 token 来源 GPU 读取输入。拥有专家的一侧知道自己接下来要计算哪些 token，可以把拉取与本地计算计划直接衔接。[S1；S2，Choosing the right communication direction]

如果用 Push，则多个来源 rank 分别写入，接收者通常需要额外知道哪些来源已完成。Pull 让“需要数据的一方”和“安排数据准备的一方”更接近，从而减少部分远端完成信号。

但不能推导为 Pull 永远快于 Push。不同访问大小、链路拓扑、TMA、访存发起能力和读写事务效率都会改变结果。论文选择的是针对其阶段、缓冲布局和 NVL72 的组合。

3.2 前向 combine 用 Push

专家计算完后主动把结果写回原 token 所在位置。与 dispatch 相反，此时生产者最清楚结果何时就绪，主动推送更容易和流水阶段连接。[S1，Fig.1；S2]

把方向选择抽象为一个原则：**尽量让能自然观察到数据就绪的执行者负责推进下一步，减少额外的跨 rank 完成协议**。

不过内存可见性、producer-consumer 顺序和最终归约仍然要被保证。“减少信号”不是删除所有 fence，更不是允许消费者读取未完成的数据。

4 SM 分工与 minibatch 流水

设总 SM 数为 S_{tot}，分给通信 S_c，计算用 $S_{\text{tot}} - S_c$。若把工作切成 m 个 minibatch，稳定阶段可用以下教学模型近似：

$$T_{\text{steady}} \approx \max\left(t_{\text{dispatch}} + t_{\text{combine}}, t_{\text{FFN}}\right).$$

通信项相加是因为它们通常竞争通信 SM 和链路，不应默认完全独立。真实实现可能在更细阶段穿插，这里只说明 pipeline 的理想节拍。

总时间还要加 fill/drain、shared expert、同步和边界：

$$T_{\text{total}} \approx T_{\text{fill}} + (m-1)T_{\text{steady}} + T_{\text{drain}}.$$

小 minibatch 提供更多重叠机会，却会增加调度与同步次数，且 GEMM tile 利用率可能下降。大 minibatch 的 GEMM 更饱满，但通信暴露窗口和尾部等待会增加。调参目标因此不是最大化 kernel 内并发度，而是最小化完整 MoE 时间。

公开接口分别暴露 forward/backward 的通信 SM 数，因为两条路径的 FLOPs、梯度归约和通信字节不同。README 建议扫描通信 SM 4–52、minibatch 2,048–16,384 等范围；这些是调参起点，不是适合所有机器的默认最优值。[S3，Config]

5 backward：把 dgrad 放在关键路径，wgrad 延后

对线性层 $Y = XW$，有

$$dX = dY\,W^T, \qquad dW = X^T dY.$$

dX 要送回前一层或远端输入，属于传播关键路径；dW 只需在相应更新前完成，有一定延后空间。MoK 先推进 expert dgrad 与 reverse-dispatch，再完成 wgrad，使后者覆盖最后一部分通信。[S2，backward schedule]

另外，wgrad 的归约维度是 token 轴。跨 minibatch 任意顺序做原子累加，会改变浮点顺序。作者将它与确定性需求共同考虑，而不是只为更高并发把所有 partial gradients 任意相加。[S2]

5.1 路由混合权重梯度，不等于路由投影也被融合

令专家内部激活为 h，down projection 为 W_2，路由权重为 p，则该分支输出为 pW_2h。对上游梯度 g：

$$\frac{\partial \mathcal{L}}{\partial p} = g^T W_2 h = h^T (W_2^T g).$$

因此可以使用 SwiGLU 激活 h 与**未乘 p 的 down-projection 输入梯度** $W_2^T g$ 的内积，计算混合权重梯度，避免长期保存完整的 down-projection 输出。[S2，Router weight gradient computation]

这里要注意缩放约定：若手中是已经乘 p 的 $d_h = pW_2^T g$，不能不加处理直接代入上式。实现必须在合适的融合点计算该内积。

MoK 返回 router weight gradients,外层 autograd 再把它们传回 router 的概率与投影计算。**能算 d_router_weights，不代表前向 Router Projection 和 Top-K 已被放进主体 Megakernel**。[S3，functional backward 返回值]

6 Ring buffer：动态路由不再要求 host 读取尺寸

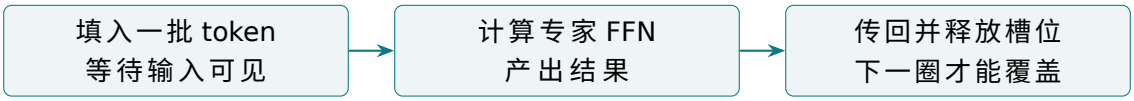
6.1 Dropless 下的动态接收量

每个 rank 收到多少路由 token 事先未知。按实际尺寸分配需要 CPU 看到 GPU counts；按最坏情况分配大 buffer 又可能严重浪费显存。丢弃超额 token 则会改变训练语义。

MoK 用固定大小的 token ring buffer，称为 macrobatch buffer。GPU 依照 schedule 分多轮处理，再循环复用槽位。逻辑工作量动态，但地址与可用空间稳定。[S1，ring token buffer；S2]

6.2 槽位的生命周期必须闭合

一个槽位经历 dispatch 填入 → FFN 消费 → combine 传出 → 可以覆盖。只有最后一个消费者完成，下一轮 dispatch 才能写进去。为了避免每一圈末尾完全排空，MoK 将上一圈对应 minibatch 的 combine 与下一圈的 dispatch 交错，让已排空的槽尽早复用。[S2]



这比“预分配大 tensor”更难的地方，在于并发状态机的正确性。少一个依赖可能覆盖未读数据，多一个全局屏障又会失去重叠收益。

6.3 Ring 带来的反向重算

前向循环会覆盖部分激活，反向需要重放相应前向，恢复 up/gate projection 与 SwiGLU 结果。无需重跑 down projection 的前向输出，因为反向所需的是它的输入和上游梯度。[S2，Reversed ring]

作者采用 reversed ring 顺序，使保留下来的激活区域尽量饱满，减少需重放的部分。公开 API 的 forward context、recompute_forward_context 和 backward 明确反映了这种生命周期。[S3]

因此，正确说法是“用可控重算换取有界缓冲与更少 host 同步”，不是“ring buffer 无成本解决无限动态 token”。

7 确定性、MXFP8 与 CLC 的真实含义

7.1 确定性不等于跨任意配置的批次不变性

作者设计固定浮点运算顺序，保证相同输入在支持的执行配置下，不随硬件调度和指令发射顺序而改变结果。[S1；S2，Determinism]

但相同输入重跑的 deterministic，与改变 batch composition 后的 batch-invariant，是不同要求。改变 M、minibatch size、量化分组、rank 数或编译策略，都可能改变数学分组与舍入位置。不能从 README 的 deterministic 一词，直接推出所有这些跨配置变化都逐位相同。

同样，MoK 不能代替 rollout 与训练端的 router replay，也不能使两个不同模型实现的全部 logits 自动一致。它覆盖的是自身计算与通信归约路径的确定性设计。

7.2 MXFP8 仍有主体外的数据准备

公开 functional API 接收 BF16 activation，权重需要预先量化，必要时准备转置视图；README 刻意把权重量化留在主体之外，以便与 FSDP 等框架安排结合。[S3，MXFP8]

因此，完整训练计时应考虑权重量化是否被缓存、是否随 optimizer step 更新、额外 scale/转置布局占用多少内存。只测主体 GEMM 时间，可能漏掉训练链上的实际成本。

7.3 CLC 解决 Megakernel 的可让出性问题

一个长期驻留的 kernel 若霸占所有 SM，会阻碍高优先级通信。作者采用 Blackwell 的 Cluster Launch Control 组织持久任务，使其更自然地为高优先级 stream 的工作让出资源，从而支持外层 RDMA overlap。[S2，CLC]

这不意味着 MoK 自己把所有跨机通信统一做进一个 kernel。NVLink 域内的 MoE 执行，与域外的数据／流水通信仍需清楚区分。

8 性能结果及其适用条件

作者的单层测试包括 scheduling、dispatch、expert FFN、combine 与最终加权合并；在单个 NVL72 rack 内使用 EP64，每 GPU 路由前 2,048 token，分别测 BF16/MXFP8 的前后向。[S2, Speedups；S3]

口径	相对测试中最快公开基线的最高加速
MXFP8 forward	2.37×
MXFP8 backward	1.78×
BF16 forward	1.92×
BF16 backward	1.58×
内部 512 GPU 生产训练 TPS	1.41×，对比原 DeepEP 路径

最后一行是不同层次的实验：512 GPU 分布在多个 NVL72 rack，不等于一个 EP512 的统一 NVLink 域。它对比内部完整训练栈，不是上述公开单层基线的简单延伸。[S2]

8.1 用 Amdahl 定律检查外推是否合理

若 MoE 占总时间 f，局部提速 a，整体理想加速为

$$A = \frac{1}{(1-f) + f/a}.$$

例如 f=0.5、a=2.37 时，A 约为 1.41。这只是说明单层 2.37× 与整模型 1.41× 在数量级上可以并存；**不是倒推作者生产模型恰有 50% 时间花在 MoE**。不同模型和测量点不应强行拼接。

局部峰值也不代表平均值。应按模型 shape、top-K、专家数、token 分布、精度、SM 配额分别看曲线，不能只取最大倍数作为稳定预期。

9 源码接口与调试顺序

官方提供 ops 层与 functional 层。前者直接调用 CUDA 实现；后者管理 workspace、schedule 与前后向上下文。生产集成建议先读 functional，理解一次执行需要保存什么，再下钻具体 kernel。[S3]

建议按 `MoKConfig → workspace → build_schedule → forward → forward_context → backward` 顺序阅读。必须记录 EP group、对称内存、shape、buffer ownership 和 autograd 保存对象，避免仅把 forward 包进 Python Module 就认为完成训练集成。

其中 `schedule_capacity_multiplier` 控制 schedule 的最坏接收占比假设，设得过小可能无法覆盖早期极端偏斜；它主要影响调度工作量，不应误解成按比例节省主要激活内存。macrobatch size 则直接影响 ring 容量与重放次数。[S3]

当前 README 的硬件前提是 Blackwell SM100/SM103、CUDA 13 系列与匹配的 PyTorch 构

建。SM100 默认需要显式选择构建架构；H200 不是该公开实现的官方目标。单机 B200 可以研究局部机制，但不能把 8 卡结果等同于 NVL72 的可扩展性验证。[S3，Setup]

10 与 MoonEP 的关系及最值得研究的实验

MoonEP 主要决定 “哪些专家在哪些 rank 复制执行”，使真实 token 数平衡；MoK 主要决定 “给定任务如何把通信与计算排进同一个执行流水”。二者潜在互补，但不是直接叠加两个开关。

整合至少需要处理物理专家映射、schedule 输入格式、临时权重副本寿命、反向前的权重恢复、grad ownership 和 ring replay。复制后的多个物理执行分支，还会影响确定性归约顺序。

最小实验可以从固定 router 轨迹开始：先测算子输出和梯度；然后扫描通信 SM 与 minibatch；再加入不同偏斜；最后测整层和完整训练 step。每组记录关键路径、暴露通信、ring 重放比例、峰值显存与数值误差。

特别值得做的是**确定性约束下的调参**：最快配置与稳定浮点顺序是否冲突？同一结果在不同 batch 划分下能否保持？性能损失来自归约结构、量化，还是同步协议？这些问题比只跑一个 “最大 TFLOPS” 更接近实际训练需求。

11 原始资料与结论

[S1] 原论文。 Mixture-of-Kittens, arXiv:2609.36070v1。重点：Fig.1 的边界、通信方向、流水、ring replay 与实验设置。

[S2] 作者技术文章。 Cursor: Mixture-of-Kittens。包含前后向调度、确定性、CLC、router weight gradient 与生产测量说明。

[S3] 官方 README。 cursor/mixture-of-kittens。本次核对 README 内容哈希 `94222e6ba01323863e4fc645eb1ee0fc863cf8b2`。源码入口为 `mok/ops.py` 与 `mok/functional.py`。

最终应记住：**MoK 的贡献是把算、传、同步和缓冲寿命放进同一个设计问题；它既不是 Attention，也不是整个训练框架**。本报告没有独立运行 NVL72 测试，性能数字均来自作者；公式解释和 Amdahl 算例属于分析。